VibeFinance: Full Build Spec

Hand this entire document to the coding agent. Build in the order of Section 15. Do not skip acceptance checks.

1. Goal and hard rules

Build a multi-agent system that takes **one ticker + one user profile** and returns a **personalized, cited, confidence-scored recommendation** with the full reasoning chain visible. Scope: **18 Nifty 50 stocks**. Product framing: decision support, never buy/sell calls (show a SEBI-style disclaimer).

Non-negotiable rules:

1. Indicators and signals are computed by **deterministic Python** (pandas), never by the LLM. The LLM only explains and synthesizes.
2. Every agent returns the same Pydantic `AgentOutput` contract.
3. **No recommendation is emitted without at least one citation.** If none exists, output 'insufficient evidence, wait'.
4. Demo runs from a **frozen snapshot** by default; Live mode is a toggle. The app must never crash on a failed feed.
5. Everything runs with `docker compose up`.
6. Simulated/synthetic data is labeled as such in the UI and README.

2. Tech stack

- Python 3.11, FastAPI, SQLAlchemy 2.x (async), Alembic, Pydantic v2
- PostgreSQL 16 + pgvector
- LangGraph (orchestration), LangSmith (tracing), LangChain (retrievers/loaders only)
- LLM: hosted API primary (configurable via env), **Ollama fallback** (e.g. llama3.1:8b)
- Embeddings: `BAAI/bge-small-en-v1.5` via sentence-transformers (local, 384 dims)
- Sentiment: FinBERT (`ProsusAI/finbert`) locally, LLM fallback
- Indicators: `pandas` , `pandas-ta` or `ta`
- Data: `yfinance` , `feedparser` , `httpx` , `beautifulsoup4` , `pypdf`
- Auth: none for the MVP (no login, no JWT); the active demo persona is sent in the header `X-User-Id`

- MCP: FastMCP server exposing tools (optional, Phase 6)
- Frontend: Next.js 14 (App Router) + Tailwind + Recharts + lightweight-charts + `native fetch (no auth library)`
- Infra: Docker Compose (api, db, web, optional ollama)

3. Repository layout

```
vibefinance/
  docker-compose.yml  .env.example  README.md
  docs/ARCHITECTURE.md  docs/DATA_ASSUMPTIONS.md
  backend/
    app/
      main.py  config.py  deps.py (get_current_user reads X-User-Id)
      db/ (base.py, models.py, session.py)  alembic/
      schemas/ (agent.py, api.py, profile.py)
      data/ (provider.py, live.py, cached.py, news.py, flows.py)
      features/ (indicators.py)
      agents/ (base.py, momentum.py, volume.py, sentiment.py,
               filings_rag.py, flow.py, conflict.py, risk.py, synthesis.py)
      rag/ (ingest.py, chunker.py, retriever.py, grader.py, crag.py)
      graph/ (state.py, build.py, runner.py)
      eval/ (metrics.py, backtest.py, calibration.py)
      api/ (routes_analysis.py, routes_profile.py, routes_portfolio.py,
            routes_metrics.py, routes_admin.py)
      mcp_server.py
    scripts/ (seed_demo.py, generate_synthetic_docs.py, fetch_snapshot.py,
run_backtest.py)
    data_snapshot/ (prices/*.csv, news/*.json, flows.csv, filings/*.md|pdf)
    tests/
  frontend/ (Next.js app)
```

4. Data layer

4.1 DataProvider interface

```
class DataProvider(Protocol):
    def get_prices(ticker, period='1y') -> DataResult[DataFrame]
    def get_news(ticker, days=14) -> DataResult[list[Headline]]
    def get_flows(days=30) -> DataResult[DataFrame]

class DataResult(BaseModel):
    data: Any
    source: Literal['live','cache','synthetic']
    as_of: datetime
    stale: bool
    error: str | None
```

`ResilientProvider` wraps Live then Cached: try live with a 5s timeout, on any exception fall back to cache, set `stale=True`, and record the error. Flags in `app/config.py` and per-request overrides: `data_mode` (cached|live), `simulate_feed_down`.

4.2 Sources

Data	Live source	Snapshot (default)
OHLCV	`yfinance` (`RELIANCE.NS`, etc.)	`data_snapshot/prices/<TICKER>.csv`, 2 years daily, fetched once by `fetch_snapshot.py`
News	Google News RSS query `<company> stock India`, ET Markets / Moneycontrol RSS via `feedparser`	`news/<TICKER>.json`, 15-30 headlines each, with timestamps and URLs
Filings	BSE corporate announcements (best-effort scrape of 3-5 companies)	**Synthetic filings and transcripts**, 40-60 docs
FII/DII	NSE daily provisional data (best-effort)	`flows.csv` (date, fii_net, dii_net), hand-made or scraped once

Live scraping is best-effort. Only prices (yfinance) and news (RSS) must work live. Filings and flows are snapshot-only in the MVP.

4.3 Watchlist universe (18)

RELIANCE, TCS, INFY, HDFCBANK, ICICIBANK, SBIN, ITC, HINDUNILVR, LT, BHARTIARTL, KOTAKBANK, AXISBANK, MARUTI, SUNPHARMA, TATAMOTORS, ADANIENT, ASIANPAINT, WIPRO. Store sector mapping in a `stocks` table.

4.4 Synthetic filings (critical for the demo)

`generate_synthetic_docs.py` uses the LLM to write, per company, 2-4 documents: quarterly results note, earnings call transcript excerpt, SEBI-style disclosure (shareholding, pledge, board change, regulatory notice). Each doc is 400-900 words, saved as markdown with front-matter: `ticker, doc_type, date, title, source_label: 'SYNTHETIC'`.

Planted scenarios (deterministic, hand-specified in the prompt):

- **ADANIENT**: promoter pledge increased from 12% to 19%; auditor remark on related-party transactions (negative).
- **TATAMOTORS**: margin guidance cut citing commodity costs (negative) while price momentum is strong (creates a conflict case).
- **INFY**: strong deal wins, raised guidance (positive).
- **HDFCBANK**: stable NIM, no red flags (neutral/positive).
- **SUNPHARMA**: regulatory observation from a foreign regulator (negative).
- **ITC**: clean, steady (neutral).
- **One ticker with NO filings at all** (e.g. WIPRO) to demo the missing-filing path.

Also set up planted price/volume anomalies by choosing snapshot windows where real volume spikes occurred; if none, the snapshot generator may inject a documented volume spike on the last trading day for ADANIENT and TATAMOTORS. Document any injected data in `DATA_ASSUMPTIONS.md`.

5. Database schema (PostgreSQL + pgvector)

Enable `CREATE EXTENSION vector;`. Use Alembic migrations.

```
stocks(ticker PK, name, sector, yf_symbol)
users(id PK = persona slug e.g. 'meena', display_name, created_at)
user_profiles(user_id FK PK, persona_label, risk_tolerance
enum[conservative,moderate,aggressive],
  horizon_years int, max_single_stock_pct numeric, sector_limits jsonb,
```

```
  loss_aversion numeric 0-1, overtrading_score numeric 0-1, updated_at)
holdings(id PK, user_id FK, ticker FK, qty numeric, avg_price numeric,
updated_at)
watchlist(user_id FK, ticker FK, PK(user_id,ticker))
behavior_events(id PK, user_id FK, ticker, action enum[buy,sell], price, qty,
  event_ts, context text, tag enum[panic_sell,fomo_buy,normal])
decisions(id PK, user_id FK, ticker, session_id FK, recommendation, followed
bool null, created_at)
documents(id PK, ticker, doc_type, title, doc_date, source_label, url,
raw_text, ingested_at)
doc_chunks(id PK, document_id FK, chunk_index, page int null, text,
  embedding vector(384), metadata jsonb)   -- ivfflat/hnsw index on
embedding, btree on (ticker)
sessions(id PK uuid, user_id FK, ticker, data_mode, flags jsonb, started_at,
finished_at)
agent_runs(id PK, session_id FK, agent, signal, confidence, status,
latency_ms,
  reasoning, output_json jsonb, created_at)
evidence(id PK, agent_run_id FK, doc_chunk_id FK null, source_type
enum[filing,news,price,flow],
  title, snippet, url, page, relevance_score)
recommendations(id PK, session_id FK, action
enum[accumulate,hold,avoid,wait,reduce],
  position_size_pct numeric, confidence, summary, reasoning_chain jsonb,
  concentration_before, concentration_after, conflict_flag bool,
degraded_flag bool)
session_metrics(id PK, session_id FK, metric_name, metric_value, unit,
created_at)
backtest_results(id PK, agent, ticker, horizon_days, n_signals, hit_rate,
avg_fwd_return, run_at)
```

Seed script creates 3 demo personas with holdings and behavior events (Section 9).

6. Agent contract

```
class Evidence(BaseModel):
    source_type: Literal['filing','news','price','flow']
    source_id: str            # doc_chunk id, url, or 'price:RELIANCE:2025-
01-10'
    title: str
    snippet: str              # exact text used
    url: str | None = None
```

```
    page: int | None = None
    relevance: float | None = None

class AgentOutput(BaseModel):
    agent: str
    ticker: str
    signal: Literal['bullish','bearish','neutral','anomalous','no_data']
    confidence: float         # 0-1
    reasoning: str            # 2-4 sentences, LLM-written from computed
facts
    metrics: dict[str, float] # raw computed values (rsi, zscore, etc.)
    evidence: list[Evidence]
    latency_ms: int
    status: Literal['ok','degraded','failed']
    notes: list[str] = []     # e.g. 'stale data', 'no filings found'
```

Every agent is wrapped by `safe_run()`: times execution, catches exceptions, returns `status='failed'`, `signal='no_data'`, `confidence=0` instead of raising.

7. Agents (specification)

7.1 Momentum Agent (deterministic)

Inputs: 1y prices. Compute RSI(14), MACD(12,26,9) histogram, SMA20/50/200, 20d and 60d return, distance from 52w high. Scoring: +1 each for RSI in 50-70, MACD hist > 0 and rising, close > SMA50 > SMA200, 20d return > 0. -1 for the mirror conditions. RSI > 75 adds an 'overbought' note. Score in [-4,4] maps to signal: >=2 bullish, <=-2 bearish, else neutral. Confidence = min(0.9, 0.4 + 0.12*abs(score)), minus 0.15 if data is stale. LLM writes the 2-4 sentence reasoning from the metrics dict. Evidence: price-derived items (e.g. 'RSI 71.2 on 2025-01-10').

7.2 Volume Anomaly Agent (deterministic)

Compute volume z-score vs the 20-day mean/std, ratio to 20d average, and price change on the spike day. Signal: z >= 2 with price up is 'anomalous' + bullish bias, z >= 2 with price down is 'anomalous' + bearish bias (record the bias in metrics), else neutral. Confidence scales with min(z/4, 1) from 0.4 to 0.85. If delivery % is not available, say so in notes (don't invent it).

7.3 Sentiment Agent

Fetch the last 14 days of headlines. Score each with FinBERT (positive/negative/neutral probs). Aggregate with recency weighting (half-life 5 days). Signal: weighted score > +0.2 bullish, < -0.2 bearish, else neutral. Confidence from headline count and agreement (low if fewer than 3 headlines or high variance). Evidence = top 3 most influential headlines with URL. If FinBERT fails, fall back to LLM scoring with structured JSON output.

7.4 Filings RAG Agent (CRAG)

See Section 8. Output: signal derived from LLM reading of graded chunks (structured JSON: stance, key_risks[], key_positives[], supporting chunk ids). Every claim must reference a chunk id; post-validate that cited ids exist in the retrieved set, else drop the claim. If zero relevant chunks survive grading, return `signal='no_data'`, `status='degraded'`, note 'No supporting document found'.

7.5 Flow Agent (optional, build last)

FII/DII 5-day and 20-day net flow trend. Market-wide context only; low weight in synthesis.

7.6 Conflict Detector (deterministic)

Take all agent outputs with `status != failed` and `signal != no_data`. Map bullish=+1, bearish=-1, neutral=0, anomalous uses its bias. Conflict if at least one +1 and at least one -1 with each confidence >= 0.5. Output: `conflict: bool`, `pairs: [(agentA, agentB)]`, `disagreement_score` (stdev of weighted signals), and `explanation` text.

7.7 Risk / Personalization Agent (deterministic core + LLM wording)

Inputs: user profile, holdings, behavior history, current prices, ticker sector. Compute:

- Current weights, sector exposure, **HHI** (sum of squared weights), top-holding weight.
- Hypothetical position: what is the sector exposure and HHI after adding `x%` of this ticker (x = profile-derived cap).
- Stock volatility (annualized 60d std) vs tolerance band (conservative <= 22%, moderate <= 32%, aggressive no cap).
- Behavior flags: `overtrading_score > 0.6`, recent `panic_sell` within 30 days on a falling stock, `loss_aversion > 0.7`.

Outputs a `RiskVerdict`: `max_position_pct`, `blocked_reasons[]`, `behavioral_nudges[]`, `concentration_before/after`, `risk_fit: good|stretch|poor`. Rules (examples):

- Volatility above tolerance, or negative filing evidence with a conservative profile → `risk_fit=poor`, max position 0%.
- Sector exposure after trade > sector limit → cap position to the remaining headroom.
- Overtrading flag and the recommendation is a buy on a recent drop → add a **48-hour cooling-off** nudge.
- Aggressive profile → cap at `max_single_stock_pct`, may accept moderate volatility.

7.8 Synthesis Agent

Inputs: all AgentOutputs, ConflictReport, RiskVerdict, profile. Step 1 (deterministic): compute weighted composite. Weights: momentum 0.25, volume 0.15, sentiment 0.20, filings 0.30, flow 0.10 (renormalize over agents that are ok/degraded with data). Confidence = weighted mean confidence x (1 - 0.5*disagreement) x (0.85 if any degraded/stale) and floor/cap into [0.05, 0.95]. Step 2 (deterministic action mapping): composite and RiskVerdict map to `accumulate | hold | avoid | wait | reduce` plus `position_size_pct`. Conflict → `wait` with low confidence. Risk poor → `avoid` or `wait`. Behavioral flag → `wait` (48h). Step 3 (LLM): write a plain-English summary (<= 150 words) from the structured facts only, with inline citation markers `[1] [2]` mapped to the evidence list. Validator: reject and retry once if summary contains numbers not present in inputs; then fall back to a templated summary. Citation guard: if no evidence with a filing or news source is attached, force action `wait` and summary 'Insufficient sourced evidence'.

8. RAG pipeline (pgvector, CRAG)

Ingest (`rag/ingest.py`): load docs from `data_snapshot/filings`, chunk 600-800 tokens with 100 overlap, preserve page/section, embed with bge-small, store in `doc_chunks` with metadata `{ticker, doc_type, date, title, source_label}`.

Retrieval flow (LangGraph subgraph or function chain):

1. Build query from ticker + focus ('risks, guidance, promoter pledge, regulatory, margins').
2. Retrieve top-8 by cosine similarity, hard-filtered on `ticker`; boost docs from the last 12 months.

3. **Grade** each chunk (LLM yes/no relevance with a short reason, or a cross-encoder). Keep relevant only; record `relevance_score`.
4. If fewer than 2 relevant: rewrite the query once (LLM) and retrieve again, widening to top-12.
5. If still none: return degraded no_data (no fabrication).
6. Generate the structured analysis only from kept chunks. Return evidence with `chunk_id, title, page, snippet, relevance`.

Log `retrieval_relevance = relevant_kept / retrieved` as a session metric.

9. Personalization demo personas

Seed three users with fixed IDs:

1. **Meena, conservative retiree**: horizon 3y, max single stock 8%, banking 40% / FMCG 30% / IT 10%, low volatility tolerance, overtrading 0.1.
2. **Arjun, aggressive young trader**: horizon 10y, max single stock 15%, IT/mid-cap heavy, high volatility tolerance.
3. **Rohan, overtrader**: moderate risk, overtrading 0.85, behavior_events showing 4 panic_sells in the last 60 days at losses.

Acceptance: the same ticker + same cached data produces **different action, position size, and nudges** for the three users (assert this in a test on ADANIENT and INFY).

10. LangGraph design

State (`TypedDict`/Pydantic): `session_id, ticker, user_id, flags, prices_df_ref, features, agent_outputs: dict, conflict, risk, recommendation, errors, timings`.

Nodes: `ingest` → `features` → fan-out to `momentum`, `volume`, `sentiment`, `filings_rag`, `flow` (parallel branches; reducers merge into `agent_outputs`) → `conflict` → `risk` → `synthesis` → `evaluate_and_log`.

Requirements: per-node latency recorded; per-node timeout (e.g. 25s for LLM nodes); a failed node never aborts the graph; LangSmith tracing enabled via env; provide a sequential runner for the parallel-vs-sequential latency comparison metric. The runner streams step events (SSE) so the UI can show the reasoning timeline live.

11. Graceful degradation (UI-triggerable)

1. **Feed down** (`simulate_feed_down`): price provider raises → fall back to cache, `stale=True` , momentum/volume confidence -0.15, banner 'stale data as of <date>'.
2. **Missing filing** (use WIPRO or `simulate_missing_filing`): filings agent returns `no_data` ; excluded from the confidence calculation; UI shows 'No supporting document found'; the recommendation must still cite news or price evidence or force `wait` .
3. **Conflicting agents** (use TATAMOTORS): conflict detector fires; synthesis emits `wait` , shows both sides and why, low confidence.
4. **LLM down**: switch to Ollama; if that fails, use templated summaries from structured facts.

Each path must be covered by an automated test asserting no exception, a valid response schema, and at least one citation or a forced `wait` .

12. Metrics and evaluation

Per session, write to `session_metrics` : `latency_<agent>_ms` , `latency_total_ms` , `latency_sequential_est_ms` , `concentration_before` , `concentration_after` , `retrieval_relevance` , `agents_ok_count` , `degraded_flag` .

Backtest (`eval/backtest.py` , CLI + admin endpoint): for each ticker and each trading day over the last ~18 months of the snapshot (using only data up to that day, no look-ahead), run the momentum and volume agents' deterministic logic; compare signal to the 30-day forward return. bullish hit = fwd return > 0, bearish hit = < 0. Store hit rate and mean forward return per agent, plus n_signals. Sentiment and filings are not backtested (document why; no historical text archive). Optional: confidence calibration (bucket confidence into 5 bins, plot predicted vs actual hit rate). Report honestly even if hit rates are near 50%.

13. API endpoints (all JSON, no auth in the MVP)

```
GET  /api/health
GET  /api/stocks                        watchlist universe + latest price
GET  /api/stocks/{ticker}/prices        OHLCV for charting
POST /api/analyze                       {ticker, user_id?, data_mode,
simulate_feed_down, simulate_missing_filing} -> full result
GET  /api/analyze/stream/{session_id}  SSE of step events
GET  /api/sessions/{id}                 full stored reasoning chain
GET  /api/me/profile   PUT /api/me/profile
GET  /api/me/portfolio PUT /api/me/portfolio   (holdings CRUD) +
concentration stats
GET  /api/me/watchlist  POST/DELETE
GET  /api/me/behavior   POST (log event)
POST /api/demo/persona/{name}           switch the demo persona (dev/demo
mode)
GET  /api/documents/chunks/{chunk_id}  citation drawer content
GET  /api/metrics/session/{id}
GET  /api/metrics/backtest
POST /api/admin/ingest  /api/admin/backtest  /api/admin/refresh-snapshot
```

The `/api/analyze` response contains: signals (all AgentOutputs), conflict report, risk verdict, recommendation (action, size, confidence, summary with citation markers), evidence list, timeline of steps with latency, flags (stale/degraded), disclaimer.

Auth: none in the MVP (no login, no JWT, no user signup). The active user is one of three seeded demo personas (`meena` , `arjun` , `rohan`). The frontend sends its slug in the `X-User-Id` header (default `meena`) and the API loads that profile and portfolio. Real auth can be added after the MVP by swapping the single `get_current_user` dependency in `deps.py` .

14. Frontend (Next.js)

Single main dashboard page:

- **Left panel**: persona switcher (3 personas), watchlist, portfolio table (qty, avg, P&L), concentration gauge (HHI / top sector %), behavior flags.
- **Center**: ticker selector, candlestick chart (lightweight-charts) with volume bars, signal cards row (Momentum / Volume / Sentiment / Filings / Flow): label chip, confidence bar, expandable reasoning and metrics, status badge (ok/degraded/failed).

- **Right**: recommendation card (action, position size, confidence, summary with clickable `[n]` citations → drawer showing the exact snippet, doc title, date, 'SYNTHETIC' label, page, relevance), conflict banner when applicable, disclaimer.
- **Bottom**: reasoning-chain timeline (per-step status + latency, live via SSE) with a parallel-vs-sequential latency bar chart; session metrics panel; backtest hit-rate table.
- **Top bar**: Live/Cached toggle, 'Simulate feed down', 'Simulate missing filing', Run Analysis button, data freshness badge.
- 'Compare personas' view: runs one ticker for all three personas side by side (strongest personalization demo).
- Loading skeletons, error toasts, dark theme, responsive down to a laptop screen. No login screen: the persona switcher is the only identity control.

15. Build order with acceptance checks

1. **Foundation**: repo, Docker Compose (api, db with pgvector, web), env, Alembic migrations, health endpoint. *Check: `docker compose up` gives a healthy API and migrated DB.*
2. **Data**: `fetch_snapshot.py`, DataProvider (live, cached, resilient), synthetic doc generator, planted scenarios, seed personas. *Check: cached prices for 18 tickers load; docs exist for 17 tickers; WIPRO has none.*
3. **Signal agents**: indicators, momentum, volume, sentiment with contracts and unit tests. *Check: tests pass; planted spike is flagged anomalous.*
4. **RAG**: ingest, retriever, grader, CRAG loop, filings agent with citation validation. *Check: ADANIENT returns pledge-related cited chunk; WIPRO returns no_data without error.*
5. **Graph**: LangGraph parallel fan-out, conflict, risk, synthesis, logging; LangSmith tracing. *Check: `/api/analyze` works for any ticker; latencies logged; TATAMOTORS triggers conflict → wait.*
6. **Personalization tests**: three personas, same input, different outputs asserted.
7. **Failure paths and tests** (Section 11).
8. **Frontend dashboard**, SSE timeline, citation drawer, compare-personas view.
9. **Metrics and backtest**, metrics panel, calibration (optional).
10. **Extras if time**: FastMCP server (`get_signals`, `search_filings`, `get_portfolio`, `analyze`), flow agent, Ollama switch test.

11. **Docs and polish**: `ARCHITECTURE.md` (agent roles, contracts, decision rules, diagram), `DATA_ASSUMPTIONS.md` (what is real, cached, synthetic, injected), README with run steps, backup demo video script.

16. Demo script (5 minutes)

1. ADANIENT as Meena: show parallel agents with latencies; open the pledge citation. Result: avoid/wait.
2. Switch to Arjun on the same stock: smaller accumulate or hold with a position cap. Then Rohan: wait 48h behavioral nudge. Use the compare-personas view.
3. TATAMOTORS: conflict banner, wait, low confidence, both sides explained.
4. Toggle feed down and WIPRO missing filing: show stale/degraded banners, no crash, still cited or forced wait.
5. Metrics panel: latency parallel vs sequential, concentration before/after, retrieval relevance, backtest hit rates.

17. Definition of done

- All rubric items demonstrable: 3+ independent signal dimensions, 3+ parallel agents with Pydantic contracts, visible RAG citations, persona-differentiated outputs, dashboard with signals/synthesis/portfolio, 3+ metrics per session, e2e workflow with visible chain, 1+ degradation path, architecture doc.
- Test suite green; `docker compose up` works from a clean clone; seeded demo needs no API keys beyond the LLM key (Ollama fallback works with none).
- Every output carries a disclaimer and every recommendation a citation or an explicit 'insufficient evidence'.

18. Environment variables

`DATABASE_URL, LLM_PROVIDER, LLM_API_KEY, LLM_MODEL, OLLAMA_BASE_URL, OLLAMA_MODEL, EMBED_MODEL, LANGSMITH_API_KEY, LANGSMITH_TRACING, LANGSMITH_PROJECT, DEFAULT_DATA_MODE, DEFAULT_USER_ID, NEXT_PUBLIC_API_URL`.